Network Security Assessment & Vulnerability Scan Report

Done By: Chidubem Nnamani
Role: Junior Cybersecurity Analyst
Date: July 15, 2026

1. Target Information & Initial Recon

Before running any active scans, I did some basic reconnaissance to make sure the target website was online and to see how its domain was set up. Everything was up and running normally.

- **Target Site:** workstudio.earlycode.net
- **IP Addresses Found:** 216.198.79.1 and 64.29.17.1
- **Hosting Setup:** Vercel Cloud Edge Network

I ran a quick ping and domain lookup to confirm the system was live and responding properly before I started any deeper network scanning.

2. Port Scanning (Nmap)

I ran an aggressive Nmap scan against the target IP to find out which ports were open, what services were running on them, and if those services leaked any version info.

- **My Scan Command:** nmap -sV -sC -T4 216.198.79.1

Open Ports Found:

- **Port 21 (FTP):** This port is open but returned an unrecognized service signature. Since it accepts active connections, it needs a manual check later to see if it allows anonymous logins or has weak passwords.
- **Port 80 (HTTP):** Open on Vercel's setup. It automatically sends web traffic over to the secure HTTPS port.
- **Port 443 (HTTPS):** Open and running a Go-based web server on Vercel's network infrastructure.

Strange Things I Noticed:

During the scan, I noticed the web server handled error redirects in a unique way. When Nmap tested the site with random requests like ***/nice ports*** and lookups for backup extensions like ***/Trinity.txt.bak***, the server actively redirected the traffic instead of just throwing a standard error. This behavior is worth a manual double-check later, just to be completely sure there aren't any actual hidden folders or old backup files left exposed on the server.

3. Automated Vulnerability Scan (Nessus)

Next, I ran a basic network scan using Tenable Nessus Essentials. This scan took about 41 minutes and targeted two distinct host configurations to search for unpatched software or missing security updates.

Scan Results:

The scan finished with a completely clean record for both the raw IP address and the main domain name:

- **IP 216.198.79.1:** 0 Critical, 0 High, 0 Medium, 0 Low, 29 Informational alerts.
- **Domain workstudio.earlycode.net:** 0 Critical, 0 High, 0 Medium, 0 Low, 27 Informational alerts.

What this means:

- **No Obvious Flaws:** Nessus didn't find any software bugs, missing patches, or bad configurations that an attacker could easily exploit.
- **Everything is Standard:** Every single flag the scanner found was strictly classified as "Informational." These are just normal network behaviors, like showing that the site uses TLS 1.2, has HSTS turned on, and uses standard TCP timestamps.
- **Solid Infrastructure:** The Vercel cloud environment hosting this site has great default security settings that do a good job of stopping automated scanners.

Problems I Faced During the Scan:

A few standard warnings popped up while Nessus was running. The scanner couldn't talk to its external callback domains like r.nessus.org, so it couldn't fully complete the ***Log4j vulnerability checks***. I also noticed some minor network congestion during the test, meaning for future labs I should probably slow down the scan speed or increase the response timeouts to keep the connection stable. Finally, my local Nessus plugin database was a bit outdated, so I need to run ***Nessus-update-plugins*** before my next scan to ensure I catch any newly released exploits.

4. Web Server Scan (Nikto)

Since the site runs a web server, I used Nikto to check the HTTP headers and see if the web configuration had any security gaps.

- **Target:** http://216.198.79.1
- **Tool Used:** Nikto v2.5.0

What I Found:

- During the web server scan, Nikto flagged two missing HTTP response headers that leave the application exposed to browser level attacks. First, the site lacks an ***X-Frame-Options header***, which means it doesn't have clickjacking protection. Without this, an attacker could easily embed this website inside an invisible iframe on a malicious page to hijack user clicks. To fix this, we need to add X-Frame-Options: **SAMEORIGIN** to the server settings.

- Second, the ***X-Content-Type-Options*** header isn't set up. This opens the door for MIME-sniffing, where older browsers try to guess a file's type based on its contents rather than trusting the server's tag. This is a clear security risk because someone could upload a hidden malicious script masked as a harmless image file and trick the browser into running it. Adding X-Content-Type-Options: no sniff to the server responses completely closes this gap.

Security Defense Noticed:

Midway through the scan, Nikto started picking up headers like ***x-vercel-mitigated: challenge***, and then the connection timed out completely. This shows that Vercel's built-in defenses successfully spotted my automated scanning traffic and blocked my IP address to protect the site.

5. Web App Penetration Testing (OWASP ZAP)

Finally, I used OWASP ZAP to check how the actual web application behaves while running, and it flagged three main issues:

- **Target:** http://earlycode.net
- **Scan Type:** Standard Dev Profile with Traditional Spider

Security Vulnerabilities Found:

- **No form protection:** The web forms don't use anti-CSRF tokens. This means if a user is logged into the site and clicks a sketchy link somewhere else, that link could secretly force the site to submit actions or changes without the user knowing. *Fix: Add secure tokens to all backend forms.*

- **No script controls:** The site doesn't have a Content Security Policy (CSP) header. Because of this, the browser will blindly run any random code or script injected into the page, making cross-site scripting (XSS) attacks way easier. *Fix: Add a basic CSP header like* ***default-src 'self';*** *to block outside scripts.*

- **Leaking server details:** The server is showing an ***X-Powered-By*** tag in its web headers. This openly tells everyone exactly what software framework the backend uses, which gives attackers a massive hint on what specific exploits to look for. *Fix: Hide or turn off this header in the server configuration.*

6. Summary of Next Steps

To fix the security gaps found in these scans, here is the quick plan we need to follow:

- **Fix the web headers:** Set up a quick rule on the web server to push out those missing headers *(Content-Security-Policy, X-Frame-Options, and X-Content-Type-Options)* while completely hiding the *X-Powered-By* tag.

- **Secure the web forms:** Update the code on the website's forms to generate secure tokens so attackers can't hijack user actions.

- **Shut down or block Port 21:** Figure out if anyone is actually using that open FTP port. If it is not needed, turn it off completely. If it is required for work, use a firewall rule to block everyone except our trusted IP addresses.

Glossary of Terms (Technical Index)

- **ACL (Access Control List):** A set of rules on a router or firewall that decides which network traffic is allowed to pass through and which traffic is blocked.

- **CSP (Content Security Policy):** A security rule set by a website that tells the browser exactly which scripts and links are safe to run, helping block malicious injections.

- **CSRF (Cross-Site Request Forgery):** A type of attack where a trick link forces a logged-in user's browser to perform actions on a website without their permission.

- **DFIR (Digital Forensics and Incident Response):** The field of cybersecurity focused on investigating cybercrimes, analyzing data traces, and responding to active security breaches.

- **DNS (Domain Name System):** The internet's phonebook. It translates human-friendly website names (like `google.com`) into raw computer IP addresses.

- **DLP (Data Loss Prevention):** Security software designed to scan network data and stop sensitive files or information from being leaked outside an organization.

- **FTP (File Transfer Protocol):** An old, standard network rule used to move files from one computer to another over a network link.

- **HSTS (HTTP Strict Transport Security):** A web server command that forces web browsers to always connect using secure HTTPS, stopping them from using insecure HTTP.

- **HTTP (Hypertext Transfer Protocol):** The foundation system used to send data across the web. Standard HTTP transmits data in clear, unencrypted text.

- **HTTPS (Hypertext Transfer Protocol Secure):** The secure version of HTTP. It uses encryption to scramble web data so it cannot be read by interceptors.

- **IP Address (Internet Protocol Address):** A unique string of numbers assigned to every single device connected to a computer network, used for identification and routing.

- **JPEG (Joint Photographic Experts Group):** A standard, widely used digital image file format.

- **MAC Address (Media Access Control Address):** A permanent, unique physical serial number built directly into a device's network hardware by the manufacturer.

- **MIME Sniffing (Multipurpose Internet Mail Extensions):** A feature where a browser tries to guess a file's format by inspecting its contents rather than trusting the server's file tag.

- **NAC (Network Access Control):** A security system that checks devices trying to connect to a network, blocking unauthorized or unsafe computers from joining.

- **PCAP / PCAPNG (Packet Capture):** A standard file format used to save recorded network traffic logs for forensic analysis.

- **RCE (Remote Code Execution):** A severe type of vulnerability that allows an attacker to run their own malicious code or commands on a target computer from anywhere in the world.

- **SMTP (Simple Mail Transfer Protocol):** The standard network protocol used for sending email messages across the internet.

- **TLS / SSL (Transport Layer Security):** Cryptographic security protocols that encrypt data moving over a network to ensure total privacy and confidentiality.

- **VLAN (Virtual Local Area Network):** A method of dividing a single physical network into smaller, isolated virtual network sections to boost security.

- **XSS (Cross-Site Scripting):** A vulnerability where an attacker injects malicious code into a trusted website, forcing the site to run the script inside a victim's browser.